

GAOIRS BACKEND SYSTEM WHITE-BOX TESTING REPORT

Comprehensive Automated Unit & Integration Testing Suite (40 Test Cases)

This document contains the structural White-Box testing verification for the Geo-Aware Incident Operation & Information Reporting System (GAOIRS). All 40 unit test cases were executed across 10 core modules to evaluate backend security, geographic boundary guards, zero-padded code generation, encryption, and spatial query performance.

Module 1: Authentication Middleware & Session Authorization (5 Test Cases)

ID	Tested Code Segment	Test Description	Input Values	Actual Result	Expected Behavior	Result
TC-W001	verifyToken()	Missing Authorization header check	headers: {}	Returned HTTP 401 Unauthorized with "Missing or invalid authorization header" message	Returns 401 error and blocks request progression	Pass
TC-W002	generateToken()	Citizen JWT token generation	user_id: "usr-101", role: "citizen"	Generated valid signed JWT string with citizen user payload	Produces valid signed JWT string containing citizen claims	Pass
TC-W003	generateToken()	Responder unit ID payload encoding	unit_id: "unit-01", role: "responder"	Encoded unit_id "unit-01" into decoded JWT object	Encodes response unit identity into token payload	Pass
TC-W004	generateToken()	Admin token optional null fields	role: "admin"	Assigned null to optional unit_id and barangay_id fields	Sets optional unit/barangay fields to null safely	Pass
TC-W005	verifyToken()	Valid Bearer token verification	authorization: "Bearer valid_jwt"	Attached user payload to req.user and executed next()	Attaches user object to request context	Pass

EVIDENCES:

Figure 1: Code Snippet - Module 1 Authentication Unit Tests

```
1: import verifyToken from './authMiddleware.js';
2: import generateToken from '../utils/generateToken.js';
3:
4: describe('White-Box Unit Testing - Module 1: Authentication Middleware', () => {
26:   test('TC-W001: verify_header - missing auth header', async () => {
34:     expect(res.statusCode).toBe(401);
36:     expect(res.jsonBody.message).toBe('Missing or invalid authorization header');
37:   });
40:   test('TC-W005: login_verification - valid bearer token', async () => {
51:     expect(next).toHaveBeenCalled();
53:     expect(req.user.id).toBe('usr-101');
56:   });
57: });
```

Module 2: Role-Based Access Control (RBAC) Middleware Guards (5 Test Cases)

ID	Tested Code Segment	Test Description	Input Values	Actual Result	Expected Behavior	Result
TC-W006	requireRole()	Undefined user object guard	req.user = undefined	Returned HTTP 401 Unauthorized status	Returns 401 Unauthorized error	Pass
TC-W007	requireAdmin()	Role mismatch guard check	req.user = { role: "REPORTER" }	Returned HTTP 403 Forbidden status	Returns 403 Forbidden error	Pass
TC-W008	requireAdmin()	Admin access permission pass	req.user = { role: "ADMIN" } }	Invoked next() middleware handler	Allows request progression via next()	Pass
TC-W009	requireResponseUnit()	Response unit permission pass	req.user = { role: "RESPONSE_UNIT" }	Invoked next() middleware handler	Allows request progression via next()	Pass
TC-W010	requireReporter()	Reporter permission pass	req.user = { role: "REPORTER" }	Invoked next() middleware handler	Allows request progression via next()	Pass

EVIDENCES:

Figure 2: Execution Results - Module 2 Role Guard Tests

```
PASS src/middleware/roleMiddleware.test.js

Module 2: Role-Based Access Control (RBAC) Middleware Guards

  ✓ TC-W006: undefined_user_guard (602 ms)
  ✓ TC-W007: role_mismatch_guard (604 ms)
  ✓ TC-W008: admin_access_pass (601 ms)
  ✓ TC-W009: responder_access_pass (603 ms)
  ✓ TC-W010: reporter_access_pass (602 ms)

Test Suites: 1 passed, 1 total | Tests: 5 passed, 5 total | Time: 3.080 s
```

Module 3: Incident Management & Geofencing Controller (7 Test Cases)

ID	Tested Code Segment	Test Description	Input Values	Actual Result	Expected Behavior	Result
TC-W011	createIncident()	NIR Geofence boundary check	lat: 14.5995, lng: 120.9842 (Manila)	Returned HTTP 400 Bad Request with "Location Out of Coverage Area" message	Rejects report with 400 error	Pass
TC-W012	getIncidents()	Paginated incident list query	page: "1", limit: "10"	Formatted data array with total=1 and page=1 pagination object	Formats paginated array response	Pass
TC-W013	getIncidentById()	Invalid ID lookup handling	id: "invalid-id"	Returned HTTP 404 Not Found status	Returns 404 Not Found error	Pass
TC-W014	verifyIncident()	Non-admin verification guard	req.user = { role: "REPORTER" }	Returned HTTP 403 Forbidden with "Only admins can verify incidents" message	Blocks non-admin verification with 403	Pass
TC-W015	verifyIncident()	Invalid verification action	action: "INVALID_ACTION_TYPE"	Returned HTTP 400 Bad Request with message "Must be APPROVE, REJECT, or REQUEST_INFO"	Rejects invalid action with 400 error	Pass
TC-W016	editIncident()	Non-admin edit guard	req.user = { role: "RESPONSE_UNIT" }	Returned HTTP 403 Forbidden with "Only admins can edit incidents" message	Blocks non-admin editing with 403	Pass
TC-W017	updateIncidentStatus()	Status update to RESPONDING	status: "RESPONDING"	Updated incident status to "RESPONDING" with HTTP 200 OK	Updates status to RESPONDING	Pass

EVIDENCES:

Figure 3: Initial Script Failure Output (Intentional Non-Admin Verification Interception)

```

FAIL src/controllers/incidentController.test.js

  ● Module 3: Incident Management › TC-W014: verify_non_admin_block

    Error: expect(received).toBe(expected) // Expected: 200, Received: 403

    at Object.toBe (src/controllers/incidentController.test.js:77:26)

Test Suites: 1 failed, 1 total | Tests: 1 failed, 6 passed, 7 total | Time: 2.150 s

```

As shown on Figure 3, the initial script failed when an unauthorized REPORTER account attempted to invoke the incident verification handler expecting a 200 OK response. This proved that the backend access control layer strictly intercepts unauthorized dispatch attempts with a 403 Forbidden exception. The test assertion was updated to validate the 403 status, resolving the error.

Figure 4: Results of Second Testing (All 7 Incident Management Tests Passed)

```

PASS src/controllers/incidentController.test.js

Module 3: Incident Management & Geofencing Controller

  ✓ TC-W011: coverage_boundary - reject out-of-bounds Manila coordinates (602 ms)

  ✓ TC-W012: get_incidents_pagination - format paginated incident query (601 ms)

```

```

✓ TC-W013: get_incident_by_id - return 404 when incident ID not found (603 ms)

✓ TC-W014: verify_non_admin_block - block non-admin users from verifying (602 ms)

✓ TC-W015: verify_invalid_action - reject invalid action types (601 ms)

✓ TC-W016: edit_non_admin_block - block non-admin users from editing (603 ms)

✓ TC-W017: update_dispatch_status - update status to RESPONDING (651 ms)

```

Test Suites: 1 passed, 1 total | Tests: 7 passed, 7 total | Time: 4.260 s

Module 4: Photo Evidence Upload & Cloud Storage Services (4 Test Cases)

ID	Tested Code Segment	Test Description	Input Values	Actual Result	Expected Behavior	Result
TC-W018	uploadIncidentPhoto()	Missing file guard check	req.file = undefined	Returned HTTP 400 Bad Request with "No file uploaded" message	Returns 400 error when file is missing	Pass
TC-W019	uploadIncidentPhoto()	Image buffer upload processing	file: { originalname: "fire.png" }	Returned HTTP 200 OK with public Supabase storage URL string	Uploads buffer and returns CDN URL	Pass
TC-W020	extractPublicId()	Supabase path extraction	url: ".../public/incidents/test.png"	Extracted path string "incidents/test.png" accurately	Extracts relative storage path	Pass
TC-W021	uploadBufferToCloudinary()	Null buffer fallback guard	buffer: null	Returned null cleanly without throwing exception	Returns null safely on invalid buffer	Pass

EVIDENCES:

Figure 5: Execution Output - Module 4 Media Upload Tests

```

PASS src/controllers/uploadController.test.js

Module 4: Photo Evidence Upload & Cloud Storage Services

✓ TC-W018: missing_file_guard (602 ms)

✓ TC-W019: process_photo_upload (651 ms)

✓ TC-W020: extract_public_id (601 ms)

✓ TC-W021: null_buffer_fallback (602 ms)

Test Suites: 1 passed, 1 total | Tests: 4 passed, 4 total | Time: 2.450 s

```

Module 5: Barangay Spatial Records Controller (2 Test Cases)

ID	Tested Code Segment	Test Description	Input Values	Actual Result	Expected Behavior	Result
TC-W022	getAll()	Barangay list pagination calculation	page: "1", limit: "10"	Returned 2 records array with totalPages=1 calculation	Calculates total pages accurately	Pass
TC-W023	create()	Barangay spatial record creation	name: "San Jose", city: "Cebu"	Returned HTTP 201 Created with generated barangay_id "b3"	Returns 201 Created with new record	Pass

EVIDENCES:

Figure 6: Execution Output - Module 5 Barangay Spatial Record Tests

```
PASS src/controllers/barangayController.test.js

Module 5: Barangay Spatial Records Controller

  ✓ TC-W022: pagination_accuracy (602 ms)

  ✓ TC-W023: create_record (651 ms)

Test Suites: 1 passed, 1 total | Tests: 2 passed, 2 total | Time: 1.250 s
```

Module 6: Incident Code Generator Algorithm (2 Test Cases)

ID	Tested Code Segment	Test Description	Input Values	Actual Result	Expected Behavior	Result
TC-W024	generateIncidentCode()	Initial zero-padded code formatting	prisma.incident.count() = 0	Formatted code string "INC-2026-001" with zero-padding	Generates INC-YYYY-001 format	Pass
TC-W025	generateIncidentCode()	Sequential counter incrementing	prisma.incident.count() = 42	Incremented sequence to format "INC-2026-043"	Generates incremented zero-padded code	Pass

EVIDENCES:

Figure 7: Initial Script Failure Output (Incorrect Zero-Padding Logic Test)

```
FAIL src/utils/incidentCode.test.js

  • Module 6: Incident Code Generator > TC-W024: first_incident_code

    Error: expect(received).toBe(expected) // Expected: "INC-2026-001", Received: "INC-2026-1"

      at Object.toBe (src/utils/incidentCode.test.js:19:26)

Test Suites: 1 failed, 1 total | Tests: 1 failed, 1 passed, 2 total | Time: 1.250 s
```

As shown on Figure 7, the initial script failed when generating an incident code without zero-padding string formatting. This proved that raw database sequence counts require explicit padStart formatting to ensure consistent code lengths across systems. The generator function was refactored with padStart(3, '0'), resolving the error.

Figure 8: Results of Second Testing (Zero-Padded Code Sequence Verified)

```
PASS src/utils/incidentCode.test.js

Module 6: Incident Code Generator Algorithm (INC-YYYY-XXX)
```

```
✓ TC-W024: first_incident_code - generate zero-padded code INC-YYYY-001 (602 ms)

✓ TC-W025: incremented_code - generate incremented zero-padded code INC-YYYY-043 (651 ms)
```

Test Suites: 1 passed, 1 total | Tests: 2 passed, 2 total | Time: 1.250 s

Module 7: System Audit Logging Controller (3 Test Cases)

ID	Tested Code Segment	Test Description	Input Values	Actual Result	Expected Behavior	Result
TC-W026	logAuditEvent()	Audit event structure validation	action: "LOGIN", resource: "USER_AUTH"	Stored audit record object with action "LOGIN"	Logs audit event payload safely	Pass
TC-W027	listAuditLogs()	Paginated audit log query	page: "1", limit: "25"	Returned HTTP 200 OK with logs array and total=1 metadata	Returns 200 OK with paginated logs	Pass
TC-W028	listDistinctActions()	Distinct audit action array mapping	req: {}	Returned 3 distinct action strings array containing "CREATE_REPORT"	Returns array of action strings	Pass

EVIDENCES:

Figure 9: Execution Output - Module 7 Audit Log Tests

```
PASS src/controllers/auditController.test.js

Module 7: System Audit Logging Controller

✓ TC-W026: log_event_action (601 ms)

✓ TC-W027: list_audit_logs (602 ms)

✓ TC-W028: distinct_actions (601 ms)

Test Suites: 1 passed, 1 total | Tests: 3 passed, 3 total | Time: 1.820 s
```

Module 8: Sensitive Data Encryption Utility (3 Test Cases)

ID	Tested Code Segment	Test Description	Input Values	Actual Result	Expected Behavior	Result
TC-W029	encrypt()	AES-256-CBC string encryption	text: "Citizen Contact 09171234567"	Generated hex iv:ciphertext string formatted with colon separator	Produces encrypted iv:ciphertext string	Pass
TC-W030	decrypt()	AES-256-CBC decryption accuracy	encryptedText: "iv:ciphertext"	Decrypted text back to exact original string "Emergency Responder Location Payload"	Restores original plaintext string	Pass
TC-W031	decrypt()	Invalid format string handling	text: "invalid_format_string"	Returned null cleanly without throwing application crash exception	Returns null safely on invalid format	Pass

EVIDENCES:

Figure 10: Execution Output - Module 8 AES Encryption Tests

PASS src/utils/encryptionUtil.test.js

Module 8: Sensitive Data Encryption Utility

✓ TC-W029: encrypt_payload (601 ms)

✓ TC-W030: decrypt_payload (602 ms)

✓ TC-W031: invalid_format_fallback (601 ms)

Test Suites: 1 passed, 1 total | Tests: 3 passed, 3 total | Time: 1.810 s

Module 9: Standardized API Response Helper Utility (4 Test Cases)

ID	Tested Code Segment	Test Description	Input Values	Actual Result	Expected Behavior	Result
TC-W032	success()	Default success response formatting	no args	Returned object { success: true, data: null, message: "Success" }	Formats default success structure	Pass
TC-W033	success()	Custom payload success formatting	data: { incident_id: "INC-2026-001" }	Returned object with success=true and custom data payload attached	Attaches custom payload to success response	Pass
TC-W034	error()	Default error response formatting	no args	Returned object { success: false, error: null, message: "Request failed" }	Formats default error structure	Pass
TC-W035	error()	Custom error payload formatting	error: { code: "INVALID_CREDENTIALS" }	Returned object with success=false and custom error payload attached	Attaches error payload to response	Pass

Module 10: Database Query Pagination Utility (5 Test Cases)

ID	Tested Code Segment	Test Description	Input Values	Actual Result	Expected Behavior	Result
TC-W036	getPagination()	Default query parameter handling	query: {}	Returned object with page=1, limit=10, skip=0 default values	Returns default page: 1, limit: 10, skip: 0	Pass
TC-W037	getPagination()	Skip offset calculation	page: 3, limit: 15	Calculated skip offset value 30 ((3-1)*15) accurately	Calculates skip offset accurately	Pass
TC-W038	getPagination()	Negative page boundary guard	page: -5, limit: 10	Enforced minimum page=1 and skip=0 boundary protection	Enforces minimum page 1 and skip 0	Pass
TC-W039	getPagination()	Maximum limit cap protection	page: 1, limit: 500	Capped limit value to maximum ceiling of 100 items	Caps maximum limit to 100 items	Pass
TC-W040	buildPaginationMeta()	Total pages calculation	page: 2, limit: 10, total: 25	Calculated totalPages=3 using Math.ceil(25/10)	Calculates totalPages metadata correctly	Pass

EVIDENCES:

Figure 11: Initial Script Failure Execution Output (Uncapped Memory Limit Assertion Test)

FAIL src/utils/pagination.test.js

```
• Module 10: Database Query Pagination › TC-W039: max_limit_cap_guard
```

```
Error: expect(received).toBe(expected) // Expected: 500, Received: 100
```

```
at Object.toBe (src/utils/pagination.test.js:30:26)
```

```
Test Suites: 1 failed, 1 total | Tests: 1 failed, 4 passed, 5 total | Time: 1.620 s
```

As shown on Figure 11, the initial script failed with an assertion mismatch when requesting an uncapped query limit of 500 records. This proved that the database query pagination algorithm strictly enforces buffer boundary caps to prevent memory exhaustion and Denial-of-Service (DoS) overhead on the backend database layer. The test assertion was refactored to validate the enforced 100-item ceiling guard, resolving the error.

Figure 12: Results of Second Testing (Passed with Enforced 100-Item Ceiling)

```
PASS src/utils/pagination.test.js
```

```
Module 10: Database Query Pagination Utility Test Suite
```

```
✓ TC-W036: default_pagination_params - default query handling (602 ms)
```

```
✓ TC-W037: skip_offset_calculation - skip offset calculation (604 ms)
```

```
✓ TC-W038: negative_page_guard - enforce page 1 minimum (601 ms)
```

```
✓ TC-W039: max_limit_cap_guard - cap maximum limit to 100 items (603 ms)
```

```
✓ TC-W040: total_pages_calculation - total pages metadata calculation (652 ms)
```

```
Test Suites: 1 passed, 1 total | Tests: 5 passed, 5 total | Time: 3.120 s
```

Final Suite Execution Summary: The full GAOIRS backend White-Box unit test suite successfully executes all 40 test cases across 13 test files with a 100% pass rate. All core security, geographic boundary, database formatting, encryption, and dispatch access control logic have been verified and documented.